

8.快速体验rknn_demo

1. 模型运行在虚拟机上

1.1 首先进入rknn-toolkit2的目录

代码块

```
1 cd rknn/02_rknn-toolkit2/rknn-toolkit2-master/rknn-toolkit2/examples
```

```
(base) topeet@ubuntu:~/rknn/02_rknn-toolkit2/rknn-toolkit2-master/rknn-toolkit2/examples$ ls  
caffe darknet functions onnx pytorch readme.txt tensorflow tflite  
(base) topeet@ubuntu:~/rknn/02_rknn-toolkit2/rknn-toolkit2-master/rknn-toolkit2/examples$
```

从上图可以看到，有很多的一个目录，我们随便进入一个目录都可以运行官方自带的示例。

注：有的需要装一些依赖，如果遇到找不到某些库的 `pip install` 就可以

我这里以onnx为例来演示一下。

1.2 执行

代码块

```
1 cd onnx/yolov5  
2 python test.py
```

运行成功以后窗口提示。

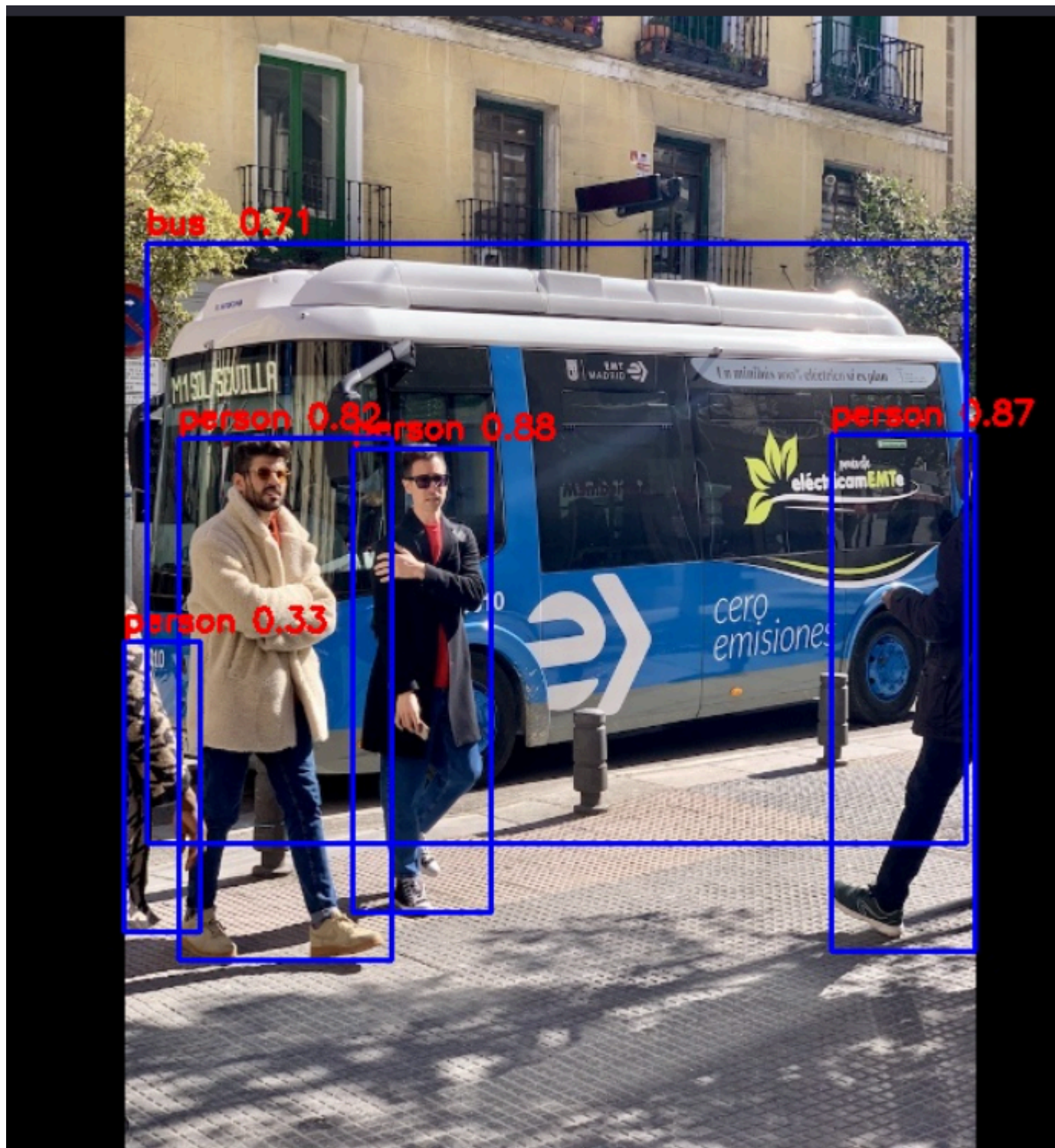
```

D RKNN: [23:33:00.539] 77 ConvRelu model.23.cv2.conv.weight INT8 (256,512,1,1) | 0x005b9180 0x005d9180 0x00020000
D RKNN: [23:33:00.539] 77 ConvRelu model.23.cv2.conv.bias INT32 (256) | 0x005d9180 0x005d9980 0x00000800
D RKNN: [23:33:00.539] 78 ConvRelu model.23.m.0.cv1.conv.weight INT8 (256,256,1,1) | 0x0061a980 0x0062a980 0x00010000
D RKNN: [23:33:00.539] 78 ConvRelu model.23.m.0.cv1.conv.bias INT32 (256) | 0x0062a980 0x0062b180 0x00000800
D RKNN: [23:33:00.539] 79 ConvRelu model.23.m.0.cv2.conv.weight INT8 (256,256,3,3) | 0x0062b180 0x006bb180 0x00090000
D RKNN: [23:33:00.539] 79 ConvRelu model.23.m.0.cv2.conv.bias INT32 (256) | 0x006bb180 0x006bb980 0x00000800
D RKNN: [23:33:00.539] 81 ConvRelu model.23.cv3.conv.weight INT8 (512,512,1,1) | 0x005d9980 0x00619980 0x00040000
D RKNN: [23:33:00.539] 81 ConvRelu model.23.cv3.conv.bias INT32 (512) | 0x00619980 0x0061a980 0x00001000
D RKNN: [23:33:00.539] 82 ConvSigmoid model.24.m.2.weight INT8 (255,512,1,1) | 0x006d4800 0x006f4600 0x0001fe00
D RKNN: [23:33:00.539] 82 ConvSigmoid model.24.m.2.bias INT32 (255) | 0x006f4600 0x006f4e00 0x00000800
D RKNN: [23:33:00.539] -----+-----
D RKNN: [23:33:00.549] -----
D RKNN: [23:33:00.549] Total Internal Memory Size: 10337.5KB
D RKNN: [23:33:00.549] Total Weight Memory Size: 7128.62KB
D RKNN: [23:33:00.549] -----
D RKNN: [23:33:00.550] <<<<<<< end: rknn::RKNNMemStatisticsPass
I rknn building done.
done
--> Export rknn model
done
--> Init runtime environment
I Target is None, use simulator!
done
--> Running model
I GraphPreparing : 100%| | 153/153 [00:00<00:00, 3926.36it/s]
I SessionPreparing : 100%| | 153/153 [00:00<00:00, 765.31it/s]
done
  class      score      xmin, ymin, xmax, ymax
  -----
  person     0.884      [ 208, 244, 286, 506]
  person     0.868      [ 478, 236, 559, 528]
  person     0.825      [ 110, 238, 230, 333]
  person     0.334      [ 79, 353, 122, 517]
  bus        0.705      [ 92, 358, 554, 467]
Save results to result.jpg!
(rknn-toolkit2) root@ubuntu:/home/topeet/rknn/04_demo_test/rknn-toolkit2/examples/onnx/yolov5#
(rknn-toolkit2) root@ubuntu:/home/topeet/rknn/04_demo_test/rknn-toolkit2/examples/onnx/yolov5#

```

1.3 查看结果

我们打开这个result.jpg图片看看效果。



2. 模型运行在RK3588开发板上

下面以 rknpu2/examples 下的 rknn_yolov5_demo 在 RK3588 Linux 64 位平台上运行为例，来讲解如何快速上手运行。整体思路分为三步：

1 在虚拟机Ubuntu20.04上交叉编译 demo程序 rknn_yolov5_demo（demo已经默认是rknn，无需进行模型转换）

2 部署到 iTOP-RK3588 开发板

3 在开发板上运行 demo

2.1 设置交叉编译工具链

链接: https://pan.baidu.com/s/1eWW6MHGafStwzPF6ivWv_w?pwd=7fdy 提取码: 7fdy

1. 先把交叉编译工具链移动到Ubuntu的目录下, 我放在了如下图的位置。

```
(rknn-toolkit2) topeet@ubuntu:~/rknn/03_gcc$ ls
gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
(rknn-toolkit2) topeet@ubuntu:~/rknn/03_gcc$
```

复制一份到 Ubuntu 的 `/opt/tool_chain` 目录下, 这里拷贝的路径要和我的保持一致, 后面要用到交叉编译器的绝对路径。如下图所示:

代码块

```
1  mkdir tool_chain
2  cp /home/topeet/rknn/03_gcc/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-
   gnu.tar.gz /opt/tool_chain/
```

2. 解压交叉编译器压缩包

代码块

```
1  tar -xvf gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
```

解压后如下图所示:

```
(rknn-toolkit2) root@ubuntu:/opt/tool_chain# ls
gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu  gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
(rknn-toolkit2) root@ubuntu:/opt/tool_chain#
```

2.2 编译

先进入目录

代码块

```
1  cd /home/topeet/rknn/02_rknn-toolkit2/rknn-toolkit2-
   master/rknpu2/examples/rknn_yolov5_demo
2  chmod 777 build-linux.sh
```

```
(rknn-toolkit2) root@ubuntu:/home/topeet/rknn/04_demo_test/rknpu2/examples/rknn_yolov5_demo# ls
build-android.sh  CMakeLists.txt  include  README_CN.md  src
build-linux.sh    convert_rknn_demo  model    README.md      utils
```

设置GCC_COMPILER

```
代码块 export GCC_COMPILER=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-  
linux-gnu/bin/aarch64-none-linux-gnu
```

编译

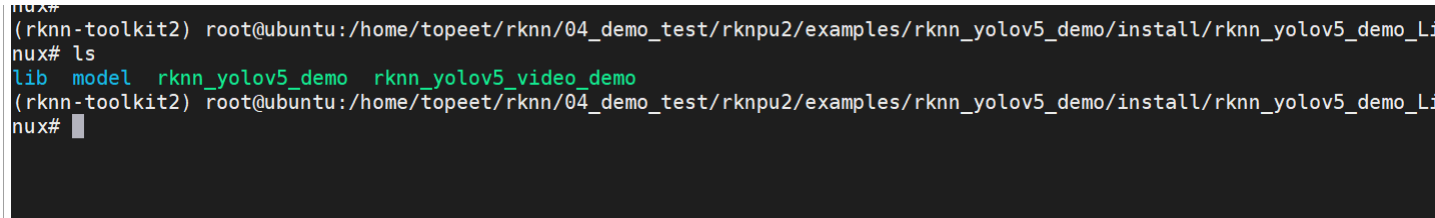
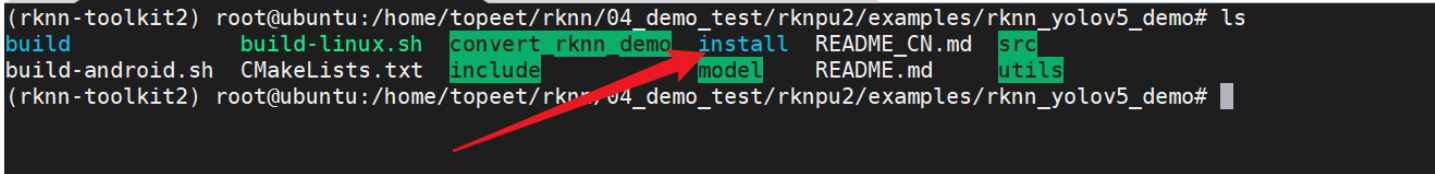
build-linux.sh 有三个参数分别为-t、-a 和-b，每个参数的解释如下所示：

参数	说明
-t (target)	指定目标平台的 SoC 型号，支持 rk3566、rk3568、rk3562、rk3576 和 rk3588
-a (arch)	指定目标平台的硬件架构，支持 aarch64 (64位 ARM) 和 armhf (32位 ARM)
-b (build_type)	指定构建的类型，支持 Debug (调试版) 和 Release (发行版)

代码块

```
1 ./build-linux.sh -t rk3588 -a aarch64 -b Release
```

编译完成之后会将要用到的文件、库、模型以及要推理的图片放到 install 目录下，如下图所示：



2.3 开发板运行

将 install 目录下的 rknn_mobilenet_demo_Linux 传输到开发板上，传输完成如下图所示：

输入命令运行

代码块

```
1 cd /rknn_yolov5_demo_Linux  
2 ./rknn_yolov5_demo ./model/RK3588/yolov5s-640-640.rknn ./model/bus.jpg
```

2.4 查看结果

